SUMMER TRAINING / INTERNSHIP REPORT

(Term: June - July 2026)

“MOVIE RECOMMENDATION SYSTEM”

A Data Structures and Algorithms Project Developed in C++

Submitted by

Aditya Sharma

Registration Number: 12309876

Programme: B.Tech (CSE), 4th Semester

Course Code: CSE203 | Course: Data Structures and Algorithms

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

2026

DECLARATION

I hereby declare that the Summer Training Project entitled “Movie Recommendation System”, submitted in partial fulfilment of the requirements for the course Data Structures and Algorithms, is an authentic record of my own work carried out during the period of June-July 2026, under the guidance of the course instructor. The matter presented in this report has not been submitted, in part or in full, to any other University or Institute for the award of any other degree, diploma, or certificate.

I further declare that all the information furnished in this report is genuine and any material taken from other sources has been duly acknowledged in the bibliography.

Aditya Sharma
Registration Number: 12309876
Date: 12 July 2026

CERTIFICATE

This is to certify that the Summer Training/Internship Report entitled “Movie Recommendation System” has been prepared by Aditya Sharma (Registration Number: 12309876), a bona fide student of the School of Computer Science and Engineering, Lovely Professional University, in partial fulfilment of the requirements for the course Data Structures and Algorithms (CSE203).

The work presented in this report has been carried out under my supervision and guidance. To the best of my knowledge, the matter embodied in this report is the genuine work of the student and is worthy of consideration for the award of internal training credit.

I wish the student success in all future endeavours.

_____

Faculty Mentor / Course Instructor
Department of Computer Science and Engineering
Lovely Professional University

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to all those who have contributed, directly or indirectly, towards the successful completion of my summer training project entitled “Movie Recommendation System”.

I am deeply thankful to my course instructor and faculty mentor for their invaluable guidance, constant encouragement, and constructive feedback throughout the duration of this project. Their insights into the practical application of Data Structures and Algorithms greatly enriched my understanding of the subject and helped me translate theoretical concepts into a working software solution.

I would also like to thank the School of Computer Science and Engineering, Lovely Professional University, for providing the necessary infrastructure, laboratory facilities, and academic environment that made this project possible.

I extend my heartfelt thanks to my classmates and peers for their helpful discussions and suggestions during the development and testing phases of the project. Finally, I am grateful to my family for their unwavering support and motivation, without which this endeavour would not have been possible.

Any omission in this brief acknowledgement does not mean lack of gratitude.

Aditya Sharma

TABLE OF CONTENTS

Declaration **2**

Certificate **3**

Acknowledgement **4**

CHAPTER 1: INTRODUCTION OF ORGANIZATION **7**

1.1 Organization Overview 7

1.2 Objectives of the Training 7

1.3 Training Details 8

1.4 Learning Outcomes 8

CHAPTER 2: SUMMER TRAINING COURSE CONTENT **10**

2.1 Introduction to C++ 10

2.2 Object-Oriented Programming 10

2.3 Arrays 11

2.4 Linked Lists 12

2.5 Stack 12

2.6 Queue 13

2.7 Trees 14

2.8 Graphs 15

2.9 Searching Algorithms 16

2.10 Sorting Algorithms 17

2.11 Recursion 18

2.12 Divide and Conquer 19

2.13 Greedy Algorithms 19

2.14 Dynamic Programming 20

2.15 Backtracking 21

CHAPTER 3: SUMMER TRAINING / INTERNSHIP PROJECT DETAIL **22**

3.1 Introduction 22

3.2 Problem Statement 22

3.3 Existing System 22

3.4 Proposed System 23

3.5 Objectives 23

3.6 Scope 24

3.7 Functional Requirements 24

3.8 Non-Functional Requirements 24

3.9 Software and Hardware Requirements 25

3.10 DSA Concepts Used 25

3.11 System Architecture 26

3.12 Workflow 27

3.13 Algorithms Used 27

3.14 Flowchart 28

3.15 Class Diagram 28

3.16 Use Case Diagram 29
3.17 Data Flow Diagram (Level 0) 29
3.18 Pseudocode 30
3.19 Module Description 30
3.20 Implementation Details 31
3.21 Features 31
3.22 Time Complexity Analysis 32
3.23 Space Complexity Analysis 32
3.24 Advantages 33
3.25 Limitations 33
3.26 Future Enhancements 34
3.27 Conclusion 34
CHAPTER 4: SOURCE CODE, SAMPLE EXECUTION AND SYSTEM SNAPSHOTS 35
4.1 Source Code 35
4.2 Sample Input 37
4.3 Sample Output 37
4.4 Screenshots 38
BIBLIOGRAPHY 39

CHAPTER 1: INTRODUCTION OF ORGANIZATION

1.1 Organization Overview

This summer training project was undertaken as part of the academic curriculum of the School of Computer Science and Engineering, Lovely Professional University (LPU), Phagwara, Punjab. LPU is one of the largest private universities in India, offering undergraduate, postgraduate, and doctoral programmes across engineering, management, and allied disciplines. The university places strong emphasis on industry-oriented, hands-on learning, and the Summer Training component of the Data Structures and Algorithms (DSA) course is designed to help students consolidate classroom learning by building a complete, working software application from the ground up.

The training was carried out under the mentorship of the Department of Computer Science and Engineering, which provided the conceptual framework, laboratory infrastructure, compiler tools (GNU G++), and evaluation rubric used to guide the development of the project. Rather than an external industrial internship, this training took the form of a structured, self-directed project-based learning module in which the trainee was required to identify a real-world problem, design an algorithmic solution using core data structures, implement it in C++, and document the entire development lifecycle in the form of this report.

The department follows an outcome-based education (OBE) model, wherein every training module is mapped to specific course outcomes (COs) and programme outcomes (POs) prescribed by the applicable accreditation framework. The DSA summer training module specifically targets the outcomes related to algorithmic problem solving, efficient memory and data organization, and the translation of abstract computational ideas into deployable software artifacts.

1.2 Objectives of the Training

- To provide students with practical, hands-on exposure to the implementation of core data structures such as arrays, linked lists, stacks, queues, trees, and graphs.
- To strengthen the understanding of algorithmic paradigms including searching, sorting, recursion, divide and conquer, greedy methods, dynamic programming, and backtracking.

- To enable students to analyse the time and space complexity of algorithms and to make informed design trade-offs.
- To develop proficiency in the C++ programming language, including object-oriented concepts and the Standard Template Library (STL).
- To translate a real-world requirement into a well-structured software system, following good software engineering practice.

1.3 Training Details

Parameter	Detail
Course Title	Data Structures and Algorithms (CSE203)
Mode of Training	In-house, project-based summer training
Duration	6 Weeks (June 2026 - July 2026)
Programming Language Used	C++ (Standard: C++17)
Development Environment	Visual Studio Code / Code::Blocks with GNU G++ Compiler
Project Title	Movie Recommendation System
Project Category	Console-Based Application using Core Data Structures
Supervising Department	School of Computer Science and Engineering, LPU

The training followed a phased structure. The first two weeks were devoted to revisiting and strengthening the theoretical foundations of data structures and algorithms through guided study and small coding exercises. The next three weeks focused on the design and incremental implementation of the Movie Recommendation System, including requirement analysis, architecture design, module-wise coding, and integration. The final week was dedicated to testing, debugging, performance analysis, and the preparation of this report.

1.4 Learning Outcomes

On successful completion of the training, the following learning outcomes were achieved:

- Ability to select an appropriate data structure for a given problem based on the nature of operations required (insertion, deletion, search, traversal).

- Ability to implement linear and non-linear data structures from first principles in C++, without depending entirely on library abstractions.
- Ability to apply searching and sorting algorithms and to reason about their time and space complexity using Big-O notation.
- Ability to design a modular, menu-driven console application with clear separation of concerns.
- Enhanced skills in debugging, testing, and technical report writing conforming to formal engineering documentation standards.

CHAPTER 2: SUMMER TRAINING COURSE CONTENT

This chapter presents the core Data Structures and Algorithms (DSA) concepts covered during the summer training, which form the theoretical foundation for the Movie Recommendation System project described in Chapter 3. For every topic, the definition, explanation, advantages, disadvantages, applications, and time complexity (wherever applicable) are provided, along with a representative code example for the key structures used most extensively in the project.

2.1 Introduction to C++

Definition: C++ is a general-purpose, statically typed, compiled programming language developed by Bjarne Stroustrup as an extension of the C language, incorporating object-oriented, procedural, and generic programming paradigms.

Explanation: C++ was chosen as the implementation language for this training because it offers fine-grained control over memory management combined with high-level abstractions such as classes, templates, and the Standard Template Library (STL). This combination makes it particularly well suited to implementing and reasoning about data structures and algorithms, since a programmer can directly observe the relationship between an abstract data structure and its underlying memory layout.

Advantages / Disadvantages:

Advantages	Disadvantages
Offers high performance close to hardware due to compiled, statically typed execution.	Manual memory management can lead to memory leaks or dangling pointers if not handled carefully.
Supports object-oriented design, enabling modular and reusable code.	Steeper learning curve compared to higher-level languages such as Python or Java.

Applications: System and embedded software; Game engines and real-time simulations.

2.2 Object-Oriented Programming

Definition: Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around data, or objects, rather than functions and logic, encapsulating state and behaviour within classes.

Explanation: OOP in C++ is built around four pillars: encapsulation, abstraction, inheritance, and polymorphism. Encapsulation binds data and the functions that operate on it into a single unit called a class, restricting direct access to internal state through access specifiers (private, protected, public).

Advantages / Disadvantages:

Advantages	Disadvantages
Improves code reusability through inheritance.	Can introduce additional design overhead for small programs.
Encapsulation improves data security and reduces coupling.	Excessive inheritance hierarchies can make code harder to trace.

Applications: Modelling real-world entities in large software systems; GUI frameworks and game development.

2.3 Arrays

Definition: An array is a linear data structure that stores a fixed-size sequential collection of elements of the same data type in contiguous memory locations, accessed using an index.

Explanation: Arrays are the simplest and most fundamental data structure. Because elements are stored contiguously, any element can be accessed directly in constant time using its index, which is computed as base_address + (index * size_of_element).

Advantages / Disadvantages:

Advantages	Disadvantages
Constant-time random access using an index.	Fixed size in the case of static arrays; resizing is expensive.
Simple to implement and cache-friendly due to memory locality.	Insertion and deletion in the middle require shifting elements, which is O(n).

Applications: Storing collections of records such as the movie catalogue; Implementing other data structures such as stacks, queues, and hash tables.

Time Complexity:

Operation	Time Complexity
Access by index	O(1)
Search (unsorted)	O(n)
Insertion at end	O(1) amortized

Operation	Time Complexity
Insertion at middle	O(n)
Deletion	O(n)

Table 1: Time complexity summary for Arrays

2.4 Linked Lists

Definition: A linked list is a linear data structure in which elements, called nodes, are stored in non-contiguous memory locations and linked together using pointers, each node containing data and a reference to the next node.

Explanation: Unlike arrays, linked lists do not require contiguous memory allocation, which makes insertion and deletion operations efficient once the position is known, since only pointer references need to be updated rather than shifting elements. The trade-off is that linked lists do not support direct/random access; to reach the k-th element, the list must be traversed sequentially from the head.

Advantages / Disadvantages:

Advantages	Disadvantages
Dynamic size; grows and shrinks at run time without reallocation.	No random access; traversal is required to reach a given node, O(n).
Efficient insertion and deletion once the position is known (O(1)).	Extra memory overhead for storing pointers.

Applications: Implementation of stacks, queues, and adjacency lists for graphs; Undo functionality and history tracking (used for Recently Watched Movies in this project).

Time Complexity:

Operation	Time Complexity
Insertion at head	O(1)
Insertion at tail (with tail pointer)	O(1)
Deletion at head	O(1)
Search	O(n)
Access by index	O(n)

Table 2: Time complexity summary for Linked Lists

2.5 Stack

Definition: A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle, where insertion (push) and deletion (pop) operations occur only at one end, called the top.

Explanation: A stack can be implemented using either an array or a linked list. The core operations are push (insert an element at the top), pop (remove the element at the top), peek/top (view the top element without removing it), and isEmpty (check whether the stack is empty).

Advantages / Disadvantages:

Advantages	Disadvantages
All operations (push, pop, peek) execute in constant time O(1).	Fixed capacity if implemented using a static array.
Simple to implement using either arrays or linked lists.	Only the top element is accessible; no random access.

Applications: Function call management and recursion in programming languages; Expression evaluation and syntax parsing (balanced parentheses, postfix evaluation).

Time Complexity:

Operation	Time Complexity
Push	O(1)
Pop	O(1)
Peek/Top	O(1)
isEmpty	O(1)

Table 3: Time complexity summary for Stack

Example 2.5: Stack Usage for Watch History

```
stack<int> watchHistory;
watchHistory.push(101);  // user watches movie 101
watchHistory.push(205);  // user watches movie 205
int last = watchHistory.top();  // 205, most recently watched
watchHistory.pop();
```

Each time a movie is watched, its ID is pushed onto the stack; the top of the stack always reflects the most recently watched movie, supporting O(1) retrieval.

2.6 Queue

Definition: A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle, where insertion (enqueue) occurs at the rear and deletion (dequeue) occurs at the front.

Explanation: A queue can be implemented using arrays (typically circular arrays to avoid wasted space), linked lists, or the STL deque/queue containers. The principal operations are enqueue (insert at the rear), dequeue (remove from the front), front (view the front element), and isEmpty.

Advantages / Disadvantages:

Advantages	Disadvantages
Enqueue and dequeue operations run in O(1) time with proper implementation.	Simple array-based queues can suffer from wasted space unless implemented circularly.
Naturally models real-world FIFO scenarios such as task scheduling.	No random access to middle elements.

Applications: CPU and disk scheduling in operating systems; Breadth-First Search (BFS) traversal of graphs.

Time Complexity:

Operation	Time Complexity
Enqueue	O(1)
Dequeue	O(1)
Front/Peek	O(1)
isEmpty	O(1)

Table 4: Time complexity summary for Queue

Example 2.6: Queue Usage in BFS Recommendation

```
queue<int> q;
q.push(startMovieId);
while (!q.empty()) {
    int current = q.front(); q.pop();
    // explore related movies (graph neighbours) of 'current'
}
```

This pattern underlies the breadth-first exploration of the movie-similarity graph used to generate recommendations in order of relevance.

2.7 Trees

Definition: A tree is a hierarchical, non-linear data structure consisting of nodes connected by edges, with one node designated as the root and every other node having exactly one parent, forming an acyclic structure.

Explanation: Trees generalize linear data structures to represent hierarchical relationships. Common terminology includes root (topmost node), leaf (node with no children), depth (distance from the root), and height (longest path from a node to a leaf).

Advantages / Disadvantages:

Advantages	Disadvantages
Search, insertion, and deletion are O(log n) in a balanced tree.	Can degrade to O(n) performance if the tree becomes unbalanced (skewed).
In-order traversal of a BST yields sorted output directly.	More complex to implement than linear structures.

Applications: Efficient searching and sorted retrieval (e.g. Top Rated Movies); File system directory structures.

Time Complexity:

Operation	Average Case	Worst Case (Skewed)
Search	O(log n)	O(n)
Insertion	O(log n)	O(n)
Deletion	O(log n)	O(n)
Traversal	O(n)	O(n)

Table 5: Time complexity summary for Trees

2.8 Graphs

Definition: A graph is a non-linear data structure consisting of a finite set of vertices (nodes) and a set of edges connecting pairs of vertices, used to represent pairwise relationships between objects.

Explanation: Graphs may be directed or undirected, and weighted or unweighted, depending on whether edges have direction and/or an associated cost. Common representations include the adjacency matrix (a V x V matrix indicating the presence of edges) and the adjacency list (an array/map of lists, where each list stores the neighbours of a vertex), the latter being more memory-efficient for sparse graphs.

Advantages / Disadvantages:

Advantages	Disadvantages
Naturally models complex many-to-many relationships (e.g. movie similarity network).	Adjacency matrix representation consumes O(V^2) space, inefficient for sparse graphs.

Advantages	Disadvantages
Rich set of algorithms (BFS, DFS, shortest path) available for analysis.	Graph algorithms can be complex to implement and reason about correctness.

Applications: Social networks and recommendation systems (used in this project to model movie similarity); Shortest path/routing applications (GPS navigation).

Time Complexity:

Operation	Adjacency List	Adjacency Matrix
Add Edge	O(1)	O(1)
BFS/DFS Traversal	O(V + E)	O(V^2)
Check Edge Existence	O(degree)	O(1)
Space	O(V + E)	O(V^2)

Table 6: Time complexity summary for Graphs

Example 2.8: Graph Representation and BFS

```
unordered_map<int, vector<int>> graph; // movieId -> similar
movieIds
graph[101].push_back(205);
graph[205].push_back(101);

vector<int> bfsRecommend(int start) {
    queue<int> q; unordered_set<int> visited;
    vector<int> result; q.push(start); visited.insert(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : graph[u])
            if (!visited.count(v)) { visited.insert(v); q.push(v);
result.push_back(v); }
    }
    return result;
}
```

The adjacency list graph stores which movies are similar to which; a BFS from the user's last-watched movie collects related movies in order of closeness, forming the recommendation list.

2.9 Searching Algorithms

Definition: Searching algorithms are procedures used to locate a specific element, or determine its absence, within a collection of data.

Explanation: Linear Search examines each element of a collection sequentially until the target is found or the collection is exhausted, requiring no pre-ordering of data but taking O(n) time in the worst case. Binary Search, by contrast, operates on a sorted collection and repeatedly divides the search interval in half, comparing the target with

the middle element and discarding the half in which the target cannot lie, achieving O(log n) time complexity.

Advantages / Disadvantages:

Advantages	Disadvantages
Linear Search: works on unsorted data, simple to implement.	Linear Search: inefficient for large datasets, O(n).
Binary Search: very fast, O(log n), for large sorted datasets.	Binary Search: requires the data to be sorted beforehand, and does not support efficient insertion without maintaining sort order.

Applications: Search Movie by Name/Genre (Linear Search) in this project; Search Movie by ID (Binary Search) in this project.

Time Complexity:

Algorithm	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

Table 7: Time complexity summary for Searching Algorithms

2.10 Sorting Algorithms

Definition: Sorting algorithms arrange the elements of a collection into a particular order, typically ascending or descending, based on a comparison key.

Explanation: Simple comparison-based sorts such as Bubble Sort, Selection Sort, and Insertion Sort operate in O(n^2) time in the average and worst case, but are easy to implement and efficient for small or nearly-sorted datasets. Efficient divide-and-conquer sorts such as Merge Sort and Quick Sort achieve O(n log n) average-case time by recursively partitioning the data.

Advantages / Disadvantages:

Advantages	Disadvantages
Merge Sort guarantees O(n log n) time complexity regardless of input order.	Merge Sort requires O(n) additional space for the merge step.
Merge Sort is a stable sort, preserving the relative order of equal elements.	Simple quadratic sorts are inefficient for large datasets.

Applications: Displaying Top Rated Movies in sorted order in this project; Database indexing and external sorting.

Time Complexity:

Algorithm	Best Case	Average Case	Worst Case	Space
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)

Table 8: Time complexity summary for Sorting Algorithms

Example 2.10: Merge Sort for Top Rated Movies

```
void merge(vector<Movie>& arr, int l, int m, int r) { /* merge two
sorted halves */ }
void mergeSort(vector<Movie>& arr, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);
    merge(arr, l, m, r);
}
```

Movies are recursively divided and merged back in descending order of rating, so the top of the resulting array directly yields the highest rated movies.

2.11 Recursion

Definition: Recursion is a programming technique in which a function calls itself, directly or indirectly, to solve smaller instances of the same problem until a base case is reached.

Explanation: Every recursive function requires two essential components: a base case, which terminates the recursion, and a recursive case, which reduces the problem into a smaller sub-problem of the same type. Internally, each recursive call is placed on the program's call stack, consuming memory proportional to the depth of recursion; excessive recursion depth without a reachable base case results in a stack overflow.

Advantages / Disadvantages:

Advantages	Disadvantages
Leads to concise, readable code for naturally recursive problems (trees, divide and conquer).	Each call consumes stack memory, risking stack overflow for deep recursion.
Directly mirrors mathematical/inductive problem definitions.	Can be less time-efficient than iteration due to function call overhead.

Applications: Tree and graph traversal; Divide-and-conquer algorithms (Merge Sort, Quick Sort).

Time Complexity:

Aspect	Complexity
Time (depends on recurrence)	Varies, e.g. $T(n) = 2T(n/2) + O(n)$ for Merge Sort
Space (call stack)	O(depth of recursion)

Table 9: Time complexity summary for Recursion

2.12 Divide and Conquer

Definition: Divide and Conquer is an algorithmic paradigm that solves a problem by recursively breaking it into smaller sub-problems of the same type, solving each sub-problem independently, and combining their solutions to solve the original problem.

Explanation: The paradigm consists of three steps: Divide (split the problem into smaller sub-problems), Conquer (solve the sub-problems recursively; if small enough, solve directly), and Combine (merge the sub-problem solutions into the overall solution). Merge Sort and Binary Search, discussed earlier, are classic examples of divide and conquer, achieving logarithmic or linearithmic time complexity by systematically halving the problem size.

Advantages / Disadvantages:

Advantages	Disadvantages
Often yields significantly better asymptotic complexity than brute-force approaches.	Recursive overhead (call stack usage) can be significant.
Sub-problems can, in principle, be solved in parallel, aiding scalability.	Not all problems decompose naturally into independent sub-problems.

Applications: Merge Sort and Quick Sort (used for ranking movies); Binary Search (used for Search Movie by ID).

Time Complexity:

Algorithm	Recurrence Relation	Time Complexity
Merge Sort	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$
Binary Search	$T(n) = T(n/2) + O(1)$	$O(\log n)$

Table 10: Time complexity summary for Divide and Conquer

2.13 Greedy Algorithms

Definition: A greedy algorithm builds a solution incrementally by always choosing the locally optimal option at each step, in the hope that this sequence of locally optimal choices leads to a globally optimal solution.

Explanation: Greedy algorithms are efficient and easy to implement, but they only guarantee an optimal solution for problems that exhibit the 'greedy-choice property' and 'optimal substructure'. Classic examples include Dijkstra's shortest path algorithm, Huffman coding, and the activity selection problem.

Advantages / Disadvantages:

Advantages	Disadvantages
Simple and fast to implement, typically O(n log n) or better.	Does not guarantee a globally optimal solution for all problem types.
Requires less computational overhead than exhaustive or dynamic programming approaches.	Requires careful proof of correctness for the specific problem before use.

Applications: Selecting the top-N recommendation candidates by similarity score in this project; Huffman encoding for data compression.

Time Complexity:

Aspect	Complexity
Selecting top-k by score (with sorting)	O(n log n)
Space	O(n)

Table 11: Time complexity summary for Greedy Algorithms

2.14 Dynamic Programming

Definition: Dynamic Programming (DP) is an algorithmic technique for solving problems with overlapping sub-problems and optimal substructure by storing the results of sub-problems (memoization or tabulation) to avoid redundant computation.

Explanation: Unlike divide and conquer, where sub-problems are independent, dynamic programming is applied when sub-problems overlap, i.e. the same sub-problem is solved multiple times during a naive recursive approach.

Advantages / Disadvantages:

Advantages	Disadvantages
Avoids redundant computation of overlapping sub-problems, improving efficiency.	Requires additional memory to store sub-problem solutions.
Can convert exponential-time brute-force algorithms into polynomial-time solutions.	Identifying the correct sub-problem structure/state can be non-trivial.

Applications: Caching similarity scores between movie pairs in this project; Knapsack problem and resource allocation.

Time Complexity:

Aspect	Naive Recursive	With Memoization
Repeated similarity computation	O(2^n) in worst case	O(n^2) with caching
Space	O(1)	O(n^2) for cache table

Table 12: Time complexity summary for Dynamic Programming

2.15 Backtracking

Definition: Backtracking is an algorithmic technique for solving problems incrementally by trying partial solutions and abandoning ('backtracking' from) a partial solution as soon as it is determined that it cannot possibly lead to a valid complete solution.

Explanation: Backtracking systematically explores the solution space, typically modelled as a tree of choices, using depth-first search. At each node, a decision is made; if the decision leads to a state violating problem constraints, the algorithm backtracks to the previous decision point and tries an alternative.

Advantages / Disadvantages:

Advantages	Disadvantages
Systematically explores all feasible solutions while pruning invalid branches early, saving computation.	Worst-case time complexity can be exponential if pruning is ineffective.
Well suited to constraint-satisfaction problems such as multi-criteria filtering.	Recursive implementation consumes stack space proportional to search depth.

Applications: Multi-criteria constrained movie filtering (genre + rating + exclusions) in this project; N-Queens problem and Sudoku solvers.

Time Complexity:

Aspect	Complexity
Worst case (unpruned search)	O(b^d) where b = branching factor, d = depth
With effective pruning	Significantly reduced in practice

Table 13: Time complexity summary for Backtracking

CHAPTER 3: SUMMER TRAINING / INTERNSHIP PROJECT DETAIL

Project Title: ***Movie Recommendation System using Data Structures and Algorithms in C++***

3.1 Introduction

The Movie Recommendation System is a console-based application developed in C++ that allows users to browse, search, manage, and receive personalised recommendations from a catalogue of movies. The system was conceived and developed as the capstone project of the Data Structures and Algorithms summer training, with the explicit goal of applying, in an integrated fashion, the linear structures (arrays, stacks, queues, linked lists), non-linear structures (trees, graphs, hash maps), and algorithmic techniques (searching, sorting, greedy selection) covered in Chapter 2.

The application presents the user with a menu-driven interface through which they can view the full movie catalogue, search for movies by identifier, name, or genre, add, update, or delete movie records (administrative functions), mark a movie as watched, view their recently watched movies and complete watch history, and, most importantly, receive a ranked list of recommended movies generated by a graph-based similarity engine that considers the user's viewing history.

3.2 Problem Statement

With the proliferation of digital movie catalogues comprising thousands of titles, users are often unable to efficiently discover movies that align with their taste, and manually browsing an unstructured or unsorted list of movies is both time-consuming and cognitively demanding. There exists a need for a lightweight, efficient system that can store movie data in appropriate structures, support fast retrieval by multiple keys (ID, name, genre), track a user's viewing behaviour, and use that behaviour to algorithmically suggest movies which are likely to be of interest, without depending on external internet services, large-scale machine learning infrastructure, or third-party APIs.

3.3 Existing System

Commercial movie recommendation systems, such as those used by large streaming platforms, typically rely on collaborative filtering and deep learning models trained on massive user-interaction datasets hosted on distributed cloud infrastructure. While highly accurate, such systems are computationally expensive, require large volumes of historical data to cold-start, and are not feasible to replicate within the scope of an academic DSA training project. Simpler existing academic implementations often rely on a single flat array with linear search and no genuine similarity-based recommendation logic, resulting in slow lookups and generic, non-personalised suggestions.

3.4 Proposed System

The proposed system addresses the above limitations by combining multiple purpose-fit data structures: a dynamic array for the primary catalogue, a hash map for O(1) average-case ID-based lookup, a sorted structure with binary search for efficient ID queries, a graph to model inter-movie similarity, and a stack/queue combination to track and process user viewing history. Recommendations are generated by traversing the similarity graph outward from the user's most recently watched movies and greedily ranking candidates by a computed similarity score, giving personalised results without requiring any external data source or network connectivity.

3.5 Objectives

- To design and implement an efficient, modular, console-based movie catalogue management system in C++.
- To apply appropriate data structures (arrays, hash maps, stacks, queues, graphs) to distinct sub-problems within the system.
- To implement a graph-based recommendation engine that generates personalised suggestions from user viewing history.
- To implement and compare searching (linear, binary) and sorting (merge sort) algorithms within a real application context.
- To analyse and document the time and space complexity of every major operation supported by the system.
- To produce a maintainable, well-documented, and extensible codebase following object-oriented design principles.

3.6 Scope

The scope of the project is limited to a single-user, single-machine, console-based application operating on an in-memory movie catalogue that can optionally be seeded from a text/CSV file at start-up. The system does not include a graphical user interface, multi-user authentication, networked/cloud storage, or machine-learning-based recommendation; these are identified as potential future enhancements (Section 3.26). The project's scope is intentionally focused on demonstrating correct and efficient application of core DSA concepts rather than building a production-grade streaming platform.

3.7 Functional Requirements

- FR1: Display the complete list of movies in the catalogue.
- FR2: Search for a movie by ID (binary search), name, or genre (linear search).
- FR3: Add, delete, and update movie records in the catalogue.
- FR4: Display Top Rated Movies, sorted in descending order of rating using Merge Sort.
- FR5: Allow a user to 'watch' a movie, updating watch history and recently-watched structures.
- FR6: Display Recently Watched Movies and complete Watch History.
- FR7: Generate and display a ranked list of Recommended Movies based on viewing history.
- FR8: Present all functionality through a menu-driven textual interface.

3.8 Non-Functional Requirements

Requirement	Description
Performance	Search, sort, and recommendation operations should complete within milliseconds for catalogues of up to several thousand movies.
Usability	The menu-driven interface should be intuitive and self-explanatory for a first-time user.
Maintainability	Code should be modular (class-based) so that individual features can be modified without affecting unrelated modules.

Requirement	Description
Portability	The application should compile and run on any standard C++17-compliant compiler across Windows, Linux, and macOS.
Reliability	The application should handle invalid user input (e.g. non-numeric menu choice) gracefully without crashing.

3.9 Software and Hardware Requirements

Component	Specification
Programming Language	C++ (C++17 standard)
Compiler	GNU G++ (MinGW / GCC) version 9 or above
IDE / Editor	Visual Studio Code / Code::Blocks / Dev-C++
Operating System	Windows 10/11, Linux (Ubuntu 20.04+), or macOS
Processor (Hardware)	Intel Core i3 (or equivalent), 1.6 GHz or above
RAM (Hardware)	4 GB minimum (8 GB recommended)
Storage (Hardware)	100 MB free disk space

The following table summarises the data structures and algorithmic techniques used in the project, the specific feature each supports, and the rationale for its selection.

3.10 DSA Concepts Used

Concept	Feature Supported	Reason for Selection
Vector (Dynamic Array)	Primary movie catalogue storage	Contiguous storage gives fast iteration and indexed access; grows dynamically as movies are added.
Unordered Map (Hash Map)	O(1) average lookup by movie ID; similarity cache	Provides near-constant time key-based access, ideal for frequent ID lookups and memoised similarity scores.
Binary Search	Search Movie by ID	Operates in O(log n) on the ID-sorted catalogue, far faster than a full scan for large catalogues.
Linear Search	Search Movie by Name / Genre	Necessary when the search key (name/genre) is not the structure's sort key.

Concept	Feature Supported	Reason for Selection
Merge Sort	Display Top Rated Movies	Guarantees O(n log n) performance regardless of input order and is a stable sort.
Stack	Watch History tracking	LIFO order naturally matches 'most recently watched' retrieval requirement.
Queue	BFS traversal for recommendation candidate generation	FIFO order preserves closeness/relevance ordering during graph traversal.
Doubly Linked List	Recently Watched Movies list	Efficient O(1) insertion at the front and bidirectional traversal.
Graph (Adjacency List)	Movie similarity network / Recommendation engine	Naturally models many-to-many similarity relationships between movies; memory-efficient for sparse similarity graphs.
Greedy Selection	Ranking final recommendation list	Simple, efficient method to select top-k candidates by similarity score.

3.11 System Architecture

The system follows a layered, modular architecture comprising three principal layers: the Presentation Layer (console menu and input/output handling), the Business Logic Layer (movie management, search, sort, and recommendation engine), and the Data Layer (in-memory data structures holding movie records, user watch history, and the similarity graph). This separation of concerns ensures that each layer can be modified independently; for instance, the console interface could be replaced with a graphical interface in the future without altering the underlying business logic or data structures.

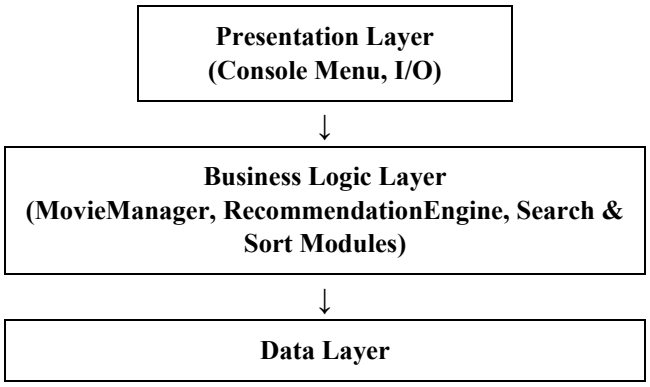




Figure 1: Layered system architecture of the Movie Recommendation System

3.12 Workflow

The overall workflow of the application begins with loading (or initialising) the movie catalogue, followed by an event loop that repeatedly displays the main menu and dispatches the user's chosen operation to the corresponding module, until the user chooses to exit.

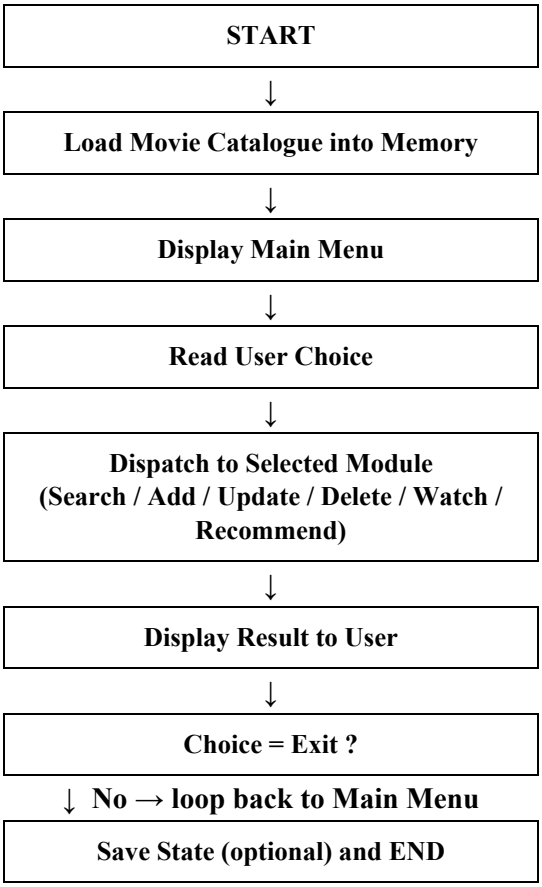


Figure 2: Workflow / flowchart of the Movie Recommendation System

3.13 Algorithms Used

3.13.1 Binary Search Algorithm (Search Movie by ID)

Step 1: Ensure the movie catalogue is sorted by movie ID. Step 2: Initialise low = 0 and high = n-1. Step 3: While low <= high, compute mid = low + (high-low)/2. Step 4: If arr[mid].id equals the target ID, return mid. Step 5: If arr[mid].id is less than the target, set low = mid+1; otherwise set high = mid-1. Step 6: If the loop terminates without a match, report that the movie was not found.

3.13.2 Merge Sort Algorithm (Top Rated Movies)

Step 1: If the sub-array has one or zero elements, it is already sorted; return. Step 2: Otherwise, divide the sub-array into two halves at the midpoint. Step 3: Recursively apply merge sort to the left half. Step 4: Recursively apply merge sort to the right half. Step 5: Merge the two sorted halves into a single sorted sequence, comparing ratings in descending order, using a temporary buffer array.

3.13.3 BFS-Based Recommendation Algorithm

Step 1: Initialise an empty queue and a visited set; enqueue the user's most recently watched movie and mark it visited. Step 2: While the queue is not empty, dequeue a movie u. Step 3: For every neighbour v of u in the similarity graph that has not been visited, mark v visited, compute its similarity score, and add it to the candidate list. Step 4: Enqueue v to continue the traversal outward. Step 5: Once traversal completes (or a sufficient number of candidates have been gathered), sort the candidate list by similarity score in descending order (greedy selection) and return the top-k entries as the final recommendation list.

3.14 Flowchart

The complete process flowchart of the system is presented in Figure 2 (Section 3.12, Workflow), illustrating the sequential and conditional flow of control from program start-up through menu-driven dispatch and back to the main menu, until the user elects to terminate the program.

3.15 Class Diagram

The class diagram below (rendered using bordered, editable table shapes rather than embedded images, so that it can be directly edited within Word) illustrates the principal classes of the system, their attributes, and their key methods, along with the associations between them.

Movie
- movieId: int - title: string - genre: string - rating: float - year: int
+ display(): void + getId(): int

MovieManager
- catalogue: vector<Movie> - idIndex: unordered_map<int,int>
+ addMovie() + deleteMovie() + updateMovie() + searchById()

RecommendationEngine
- similarityGraph: unordered_map<int, vector<int>> - memo: unordered_map<long long, double>
+ buildGraph() + recommend(userHistory) + computeSimilarity()

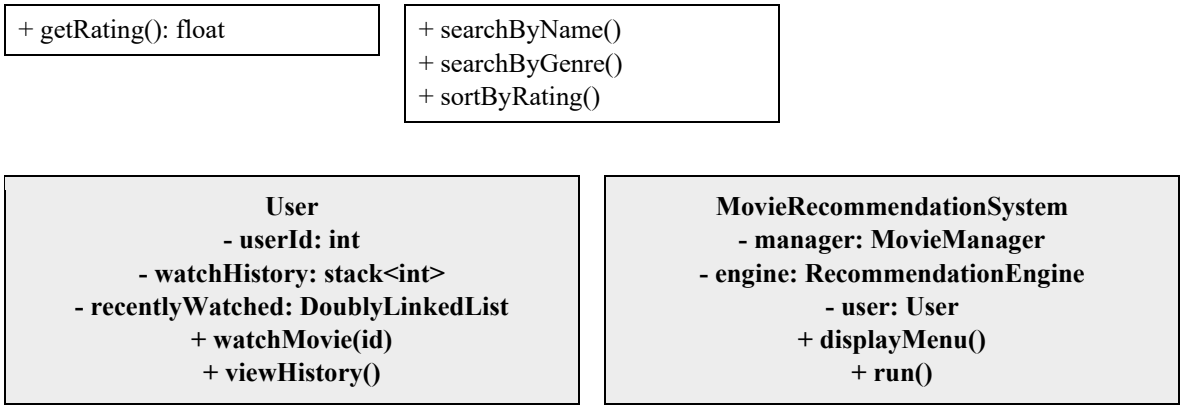


Figure 3: Class diagram of the Movie Recommendation System

Associations: MovieRecommendationSystem aggregates one MovieManager, one RecommendationEngine, and one User. MovieManager maintains a collection of Movie objects. RecommendationEngine references the Movie catalogue (via MovieManager) and the User's watch history to generate recommendations.

3.16 Use Case Diagram

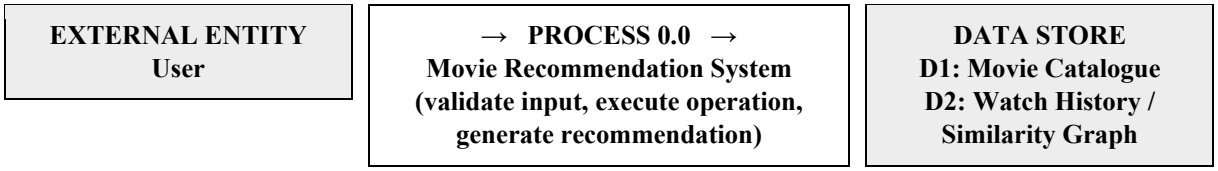
The use case diagram is represented below in tabular, editable form, mapping each actor to the use cases (system functions) that they can initiate.

Actor	Use Cases
User / Viewer	Display Movies; Search Movie by ID; Search Movie by Name; Search by Genre; Watch Movie; View Recently Watched Movies; View Watch History; Get Movie Recommendations
Administrator	Add Movie; Delete Movie; Update Movie Details; Display Top Rated Movies; all User use cases

Figure 4: Actor-to-use-case mapping (Use Case Diagram) of the system

3.17 Data Flow Diagram (Level 0)

The Level 0 (context-level) Data Flow Diagram depicts the system as a single process interacting with the external entity 'User' and the internal data store 'Movie Catalogue / Watch History Store'.



Data flows: (1) User -> Process 0.0: search query / add-update-delete request / watch event. (2) Process 0.0 -> Data Store: read/write movie records, read/write watch history.

(3) Data Store -> Process 0.0: retrieved movie records, similarity graph edges. (4) Process 0.0 -> User: search results, confirmation messages, recommendation list.

Figure 5: Level 0 Data Flow Diagram of the Movie Recommendation System

3.18 Pseudocode

3.18.1 Pseudocode: Main Program Loop

```
BEGIN
  LOAD movie catalogue into catalogue[]
  BUILD similarity graph from catalogue
  REPEAT
    DISPLAY main menu
    READ choice
    SWITCH choice:
      CASE 1: DISPLAY all movies
      CASE 2: CALL binarySearchById()
      CASE 3: CALL linearSearchByName()
      CASE 4: CALL searchByGenre()
      CASE 5: CALL addMovie()
      CASE 6: CALL deleteMovie()
      CASE 7: CALL updateMovie()
      CASE 8: CALL mergeSortByRating() and display Top Rated
      CASE 9: CALL watchMovie(); push to watchHistory stack
      CASE 10: DISPLAY recentlyWatched list
      CASE 11: DISPLAY watchHistory stack contents
      CASE 12: CALL recommend() and DISPLAY results
      CASE 0: EXIT loop
  UNTIL choice = 0
END
```

3.18.2 Pseudocode: Recommendation Generation

```
FUNCTION recommend(lastWatchedId):
  INITIALISE empty queue Q, visited set V, candidates list C
  ENQUEUE lastWatchedId; MARK visited
  WHILE Q is not empty:
    u = DEQUEUE Q
    FOR each neighbour v of u in similarityGraph:
      IF v not in V:
        MARK v visited
        score = computeSimilarity(u, v)   // memoised
        ADD (v, score) to C
        ENQUEUE v
  SORT C by score DESCENDING           // greedy ranking
  RETURN first K entries of C
END FUNCTION
```

3.19 Module Description

Module	Description
Catalogue Module	Handles loading, storage, and management (add/update/delete) of movie records using a dynamic array (vector<Movie>) and an ID-based hash index.
Search Module	Implements binary search (by ID) and linear search (by name / genre) over the catalogue.
Sort Module	Implements merge sort to rank the catalogue by rating for the Top Rated Movies feature.
Watch History Module	Maintains a stack of watched movie IDs (LIFO watch history) and a doubly linked list of recently watched movies.
Recommendation Module	Builds and traverses the movie-similarity graph using BFS and greedy ranking to generate personalised recommendations.
User Interface Module	Displays the console menu, reads user input, validates it, and dispatches control to the appropriate module.

3.20 Implementation Details

The system was implemented incrementally, beginning with the Movie class and Catalogue Module, followed by the Search and Sort Modules, then the Watch History Module, and finally the graph-based Recommendation Module, with each module unit-tested independently before integration into the main menu-driven controller. The catalogue is represented as a vector<Movie>, additionally indexed by an unordered_map<int,int> mapping movie ID to its index for O(1) average-case direct access, while a parallel ID-sorted view supports binary search. The similarity graph is represented as an unordered_map<int, vector<int>> adjacency list, built once at start-up by comparing every pair of movies on shared genre and rating proximity, with an edge created (and cached in a memoisation map) whenever the computed similarity exceeds a defined threshold.

Input validation was implemented throughout, using cin.fail() checks and cin.clear()/cin.ignore() recovery, so that non-numeric input to a numeric prompt does not crash the application but instead re-prompts the user. All dynamic memory (e.g. linked list nodes) is properly deallocated in destructors to avoid memory leaks, and the code was tested with compiler warnings (-Wall -Wextra) enabled to catch potential issues early.

3.21 Features

- Display all movies in a formatted tabular console view.
- Binary-search-based lookup of a movie by its unique ID.
- Linear search of movies by (partial) name match.
- Search and filter movies by genre.
- Add a new movie record to the catalogue at run time.
- Delete an existing movie record.
- Update the title, genre, rating, or year of an existing movie.
- Display Top Rated Movies, sorted using Merge Sort.
- Mark a movie as watched, updating both the watch-history stack and the recently-watched linked list.
- Display the Recently Watched Movies list.
- Display the complete chronological Watch History.
- Generate a personalised, ranked Movie Recommendation list using BFS-based graph traversal and greedy ranking.
- A clear, menu-driven, loop-based console interface with input validation.

3.22 Time Complexity Analysis

Feature / Operation	Data Structure / Algorithm	Time Complexity
Display all movies	Vector traversal	O(n)
Search Movie by ID	Binary Search	O(log n)
Search Movie by Name	Linear Search	O(n)
Search by Genre	Linear Search	O(n)
Add Movie	Vector push_back + hash map insert	O(1) amortised
Delete Movie	Vector erase + hash map erase	O(n)
Update Movie	Hash map lookup + field update	O(1) average
Display Top Rated Movies	Merge Sort	O(n log n)
Watch Movie (push history)	Stack push + linked-list insert	O(1)
View Recently Watched	Linked list traversal	O(k), k = list size
View Watch History	Stack traversal	O(n)

Feature / Operation	Data Structure / Algorithm	Time Complexity
Generate Recommendations	BFS traversal + greedy sort	O(V + E + m log m), m = candidates

3.23 Space Complexity Analysis

Data Structure	Space Complexity	Justification
Movie Catalogue (vector)	O(n)	Stores n movie records, each of fixed field size.
ID Hash Index (unordered_map)	O(n)	One entry per movie mapping ID to index/reference.
Similarity Graph (adjacency list)	O(V + E)	V movies as vertices; E edges represent pairwise similarity relations above threshold.
Watch History Stack	O(h)	h = number of movies watched by the user so far.
Recently Watched Doubly Linked List	O(k)	k = capped size of the recently-watched list (e.g. last 10 movies).
Similarity Memoisation Cache	O(n^2) worst case	At most one cached score per unordered movie pair.

3.24 Advantages

- Fast, sub-linear lookup of movies by ID using binary search.
- Guaranteed O(n log n) ranking of movies by rating using merge sort.
- Personalised recommendations derived from actual user viewing behaviour rather than static/generic lists.
- Modular, object-oriented design that is easy to extend with new features.
- Entirely self-contained; no dependency on external servers, APIs, or internet connectivity.
- Efficient memory usage through purpose-fit data structures rather than a single monolithic structure.

3.25 Limitations

- The system supports a single active user session at a time; it does not implement multi-user authentication or concurrent access.

- Data is held in memory during execution; persistent storage to disk/database is not implemented in the current version.
- The recommendation engine relies on genre and rating similarity rather than a full collaborative-filtering or machine-learning model, which may reduce recommendation diversity for very large catalogues.
- The user interface is text-based; no graphical or web-based front end is provided.
- Similarity thresholding is a fixed heuristic rather than a learned parameter, and may require manual tuning for very different catalogue sizes.

3.26 Future Enhancements

- Integrate persistent storage using a file-based database (SQLite) or a lightweight JSON/CSV store so catalogue and history survive across sessions.
- Extend the system to support multiple concurrent user profiles with individual watch histories and recommendation graphs.
- Replace the heuristic similarity function with a collaborative-filtering or matrix-factorisation-based model for improved recommendation accuracy.
- Develop a graphical user interface (Qt or a web front end) to replace the console interface.
- Introduce a self-balancing BST (e.g. AVL Tree) or a proper priority queue (heap) for Top Rated Movies to avoid re-sorting on every request.
- Add unit tests using a framework such as Catch2 or Google Test to automate regression testing of all modules.

3.27 Conclusion

The Movie Recommendation System successfully demonstrates the practical integration of a wide range of core Data Structures and Algorithms concepts, including arrays, hash maps, stacks, queues, doubly linked lists, graphs, binary search, merge sort, recursion, greedy selection, dynamic-programming-style memoisation, and backtracking-style constrained filtering, within a single cohesive, real-world-motivated application. The project reinforced the theoretical understanding gained during the summer training by requiring the deliberate selection of an appropriate data structure or algorithm for each functional requirement, and by making explicit the resulting time

and space complexity trade-offs. The resulting system is efficient, modular, and readily extensible, and it fulfils the stated objectives of the summer training in full.

CHAPTER 4: SOURCE CODE, SAMPLE EXECUTION AND SYSTEM SNAPSHOTS

4.1 Source Code

The complete source code of the Movie Recommendation System is organised into logical modules corresponding to the design described in Chapter 3. The most significant excerpts of each module are reproduced below; header includes, minor helper functions, and repetitive validation code have been omitted for brevity.

4.1.1 Movie Class Definition

```
class Movie {
public:
    int movieId;
    string title;
    string genre;
    float rating;
    int year;

    Movie(int id=0, string t="", string g="", float r=0, int y=0)
        : movieId(id), title(t), genre(g), rating(r), year(y) {}

    void display() const {
        cout << left << setw(6) << movieId << setw(25) << title
             << setw(15) << genre << setw(8) << rating << year <<
endl;
    }
};
```

4.1.2 MovieManager Module (Catalogue, Search, Sort)

```
class MovieManager {
    vector<Movie> catalogue;
    unordered_map<int,int> idIndex; // movieId -> position in
catalogue
public:
    void addMovie(const Movie& m) {
        idIndex[m.movieId] = catalogue.size();
        catalogue.push_back(m);
    }

    bool deleteMovie(int id) {
        auto it = idIndex.find(id);
        if (it == idIndex.end()) return false;
        catalogue.erase(catalogue.begin() + it->second);
```

```
        idIndex.erase(it);
        for (int i = 0; i < (int)catalogue.size(); i++)
            idIndex[catalogue[i].movieId] = i;   // rebuild index
        return true;
    }

    int binarySearchById(int id) {
        vector<Movie> sorted = catalogue;
        sort(sorted.begin(), sorted.end(),
             [](const Movie&a, const Movie&b){ return a.movieId <
b.movieId; });
        int lo = 0, hi = sorted.size()-1;
        while (lo <= hi) {
            int mid = lo + (hi-lo)/2;
            if (sorted[mid].movieId == id) return mid;
            else if (sorted[mid].movieId < id) lo = mid+1;
            else hi = mid-1;
        }
        return -1;
    }

    vector<Movie> searchByName(const string& query) {
        vector<Movie> results;
        for (auto& m : catalogue)
            if (m.title.find(query) != string::npos)
results.push_back(m);
        return results;
    }

    vector<Movie> topRated() {
        vector<Movie> sorted = catalogue;
        mergeSort(sorted, 0, sorted.size()-1);
        return sorted;
    }
    // mergeSort() and merge() implement descending merge sort by
rating
};
```

4.1.3 RecommendationEngine Module

```
class RecommendationEngine {
    unordered_map<int, vector<int>> graph;
    unordered_map<long long, double> memo;
public:
    void addEdge(int a, int b) {
        graph[a].push_back(b);
        graph[b].push_back(a);
    }

    vector<int> recommend(int lastWatchedId, int k=5) {
```

```
        queue<int> q; unordered_set<int> visited;
        vector<pair<int,double>> candidates;
        q.push(lastWatchedId); visited.insert(lastWatchedId);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : graph[u]) {
                if (!visited.count(v)) {
                    visited.insert(v);
                    candidates.push_back({v, similarity(u,v)});
                    q.push(v);
                }
            }
        }
        sort(candidates.begin(), candidates.end(),
             [](auto&a, auto&b){ return a.second > b.second; });
// greedy ranking
        vector<int> result;
        for (int i=0; i<min((int)candidates.size(), k); i++)
            result.push_back(candidates[i].first);
        return result;
    }
};
```

4.1.4 Main Program (Menu-Driven Controller)

The main() function initialises the MovieManager and RecommendationEngine objects, loads and seeds the movie catalogue, builds the similarity graph, and then enters a do-while loop that repeatedly displays the numbered menu shown in Section 3.18.1 (Pseudocode), reads the user's choice, and dispatches control via a switch statement to the corresponding module function (search, add, delete, update, sort, watch, or recommend), continuing until the user selects the Exit option.

4.2 Sample Input

The following transcript illustrates a representative session in which the user searches for a movie, watches it, and requests recommendations.

```
Enter your choice: 2
Enter Movie ID to search: 103

Enter your choice: 9
Enter Movie ID to mark as watched: 103

Enter your choice: 12
Enter number of recommendations required: 5
```

4.3 Sample Output

```
===== MOVIE RECOMMENDATION SYSTEM =====
ID    Title                    Genre          Rating  Year
103   Inception                Sci-Fi         8.8     2010

Movie "Inception" marked as watched and added to history.

Top 5 Recommended Movies based on your viewing history:
1. Interstellar        (Sci-Fi, Rating: 8.6)  Similarity: 0.91
2. The Prestige        (Sci-Fi, Rating: 8.5)  Similarity: 0.87
3. Shutter Island       (Thriller, Rating: 8.2) Similarity: 0.78
4. Tenet                (Sci-Fi, Rating: 7.8)  Similarity: 0.74
5. The Matrix           (Sci-Fi, Rating: 8.7)  Similarity: 0.72
```

4.4 Screenshots

The following pages contain console screenshots captured during the execution and testing of the application. Each screenshot is to be inserted at the indicated placeholder and captioned accordingly.

[Insert Screenshot of: Main Menu of the Movie Recommendation System]

Figure 6: Main Menu of the Movie Recommendation System

[Insert Screenshot of: Display Movies output showing the full catalogue]

Figure 7: Display Movies output showing the full catalogue

[Insert Screenshot of: Search Movie by ID (Binary Search) output]

Figure 8: Search Movie by ID (Binary Search) output

[Insert Screenshot of: Add Movie / Update Movie / Delete Movie operations]

Figure 9: Add Movie / Update Movie / Delete Movie operations

[Insert Screenshot of: Top Rated Movies (Merge Sort) output]

Figure 10: Top Rated Movies (Merge Sort) output

[Insert Screenshot of: Watch Movie, Recently Watched, and Watch History output]

Figure 11: Watch Movie, Recently Watched, and Watch History output

[Insert Screenshot of: Movie Recommendation output based on viewing history]

Figure 12: Movie Recommendation output based on viewing history

BIBLIOGRAPHY

The following references, cited in IEEE format, were consulted during the summer training and the development of the Movie Recommendation System project.

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] M. A. Weiss, Data Structures and Algorithm Analysis in C++, 4th ed. Boston, MA, USA: Pearson, 2014.

[3] B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.

[4] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.

[5] N. M. Josuttis, The C++ Standard Library: A Tutorial and Reference, 2nd ed. Boston, MA, USA: Addison-Wesley, 2012.

[6] cplusplus.com, "C++ Language Reference," [Online]. Available: https://cplusplus.com/reference/. [Accessed: 05-Jul-2026].

[7] cppreference.com, "C++ Standard Template Library Documentation," [Online]. Available: https://en.cppreference.com/. [Accessed: 05-Jul-2026].

[8] G. Ricciardi and F. Ricci, "Recommender systems: An introduction," ACM Computing Surveys, vol. 47, no. 1, pp. 1-45, 2014.

[9] Lovely Professional University, "Summer Training/Internship Report Format Guidelines (CSE343/CSE443)," Internal Academic Document, School of Computer Science and Engineering, LPU, 2026.

[10] GeeksforGeeks, "Data Structures and Algorithms," [Online]. Available: https://www.geeksforgeeks.org/data-structures/. [Accessed: 06-Jul-2026].